

Weekly Progress Report

Name: Md Jishan

Domain: Python Development Internship

Date of submission: 30 June 2026

Week Ending: 02

I. Overview

This week, I challenged myself to upgrade all four Python projects from basic scripts to feature-rich, real-world tools. It was exciting but also overwhelming at times. My primary focus was on adding REST APIs, PostgreSQL support, a Timer with difficulty levels, Logging, and bcrypt encryption. While I successfully coded all the features locally, I realized deployment is tougher than I thought, so I'll focus on that next week.

II. Achievements

1. Environment & Technology Upgrades

- Upgraded my development environment by installing PostgreSQL for production database needs.
- Installed additional libraries: Flask-SQLAlchemy, psycopg2-binary, gunicorn, Flask-HTTPAuth, bcrypt, and threading modules.
- Set up Render.com account for deployment and configured environment variables.

2. Python Project Contributions

Name of the project:- URL Shortener (Week 02)

- Created REST APIs for programmatic URL shortening:- POST /api/shorten – Accepts long URL via JSON, returns short code. GET /api/expand/<code> – Returns original URL with click statistics.
- Added API Key Authentication using Flask-HTTPAuth to secure the APIs.
- Added Copy-to-Clipboard Button in the web interface for better user experience.
- Added Click Counter to track how many times each short URL is accessed.
- Made the app PostgreSQL ready using Flask-SQLAlchemy.
- Tested all APIs using Postman – all endpoints work correctly.

Name of the project:- Quiz Game (Week 02)

- Added 10-second Timer for each question using Python's threading module.

- Created Three Difficulty Levels:-Created Three Difficulty Levels: Easy (10 sec, 1 pt), Medium (7 sec, 2 pts), Hard (5 sec, 3 pts).
- Added Bonus Points for answering quickly before time expires.
- Expanded Question Bank with 10 new questions covering Python and General Knowledge.
- Added High Score Database using SQLite to track top scores.
- User can now select difficulty before starting the quiz.

Name of the project:- File Organizer (Week 02)

- Refactored Code to PEP 8 Standards – added proper docstrings, comments, and snake_case naming.
- Added Custom Category Support – users can define their own file categories in config.json.
- Implemented Logging System – all file movements are recorded in file_organizer.log with timestamps.
- Added support for 30+ file extensions (.psd, ai, mp4, mkv, webp, etc).
- Added duplicate file handling – automatically renames files if a duplicate exists in the destination folder.

Name of the project:- Password Manager (Week 02)

- Upgraded Master Password Hashing from SHA256 to bcrypt for stronger security.
- Added Password Strength Checker – evaluates passwords as Weak, Medium, or Strong.
- Implemented Export/Import Functionality – users can export encrypted passwords to JSON and import them back.
- Added gitignore support to prevent accidental commit of key key file.

3. Learning Python

- Gained practical experience with REST API development using Flask.
- Understood threading for implementing timers and non-blocking input.
- Learned about bcrypt for secure password hashing.
- Practiced PostgreSQL integration with Flask-SQLAlchemy.

III. Challenges & Hurdles

1. Flask-HTTPAuth Installation Issue

Challenge:- When I tried to run the enhanced URL Shortener, I got a Module Not Found Error: No module named flask_httpauth.

Resolution:- I ran pip install flask-httpauth in the terminal. After installation, the code ran without errors. I also realized I needed **Flask-SQLAlchemy** and psycopg2-binary for PostgreSQL support.

2. Timer Implementation in Quiz Game

Challenge:- Adding a timer that interrupts the user's input without freezing the entire program was difficult. Using time.sleep() would pause everything, which wasn't the desired behavior.

Resolution:- I researched different approaches and found that using threading with a flag variable works best. I created a separate thread that counts down and sets a flag when time expires. The main thread checks this flag before processing the answer.

3. PostgreSQL Connection Error

Challenge: When testing the URL Shortener with PostgreSQL locally, I got a connection error because the database URL format was incorrect.

Resolution:- I learned that Render uses postgres:// but SQLAlchemy needs postgresql://. I added a conditional check to replace the URL format automatically. I also used environment variables to store the database URL securely.

4. Git Security Issue

Challenge:- I almost committed the key file (encryption key) to GitHub again, which is a security risk.

Resolution:- I immediately added key, db, and log to gitignore before committing. I also learned to use environment variables for sensitive data instead of storing them in code.

IV. Lessons Learned

1. **Deployment Requires Careful Planning** – Moving from SQLite to PostgreSQL taught me that production databases need proper configuration. Environment variables are essential for keeping credentials secure.
2. **Threading is Powerful but Tricky** – Implementing the timer taught me how multithreading works in Python. I now understand the difference between threading and multiprocessing.
3. **Security is Not Optional** – Using bcrypt for master passwords and properly handling encryption keys showed me that security must be a priority from the start.

4. **Version Control Discipline is Critical** – I learned the importance of .gitignore and being careful about what I commit to GitHub. Accidentally exposing an encryption key could compromise all user data.

V. Learning Resources

- Flask-SQLAlchemy & HTTPAuth Docs – For APIs and databases.
- YouTube Tutorials – Python Threading and Flask REST API.
- Stack Overflow – For the PostgreSQL connection fix.
- Postman – For testing APIs.

VI. Next Week's Goals

1. Write a complete User Manual for all 4 projects with screenshots.
2. Add unit tests using pytest.
3. Build a simple Portfolio Website to showcase the projects.
4. Actually deploy the URL Shortener on Render.com.

VII. Additional Comments

- **Total Effort:-** I spent about 5-6 hours daily on these projects this week. Most of my time went into understanding how REST APIs work because this was my very first time building them.
- **The Real Struggle:-** The PostgreSQL connection error took me almost 2 hours to fix because I didn't know Render uses a different URL format. I almost gave up, but Stack Overflow saved me.
- **My Favorite Moment:-** When I tested the API in Postman and it returned the correct JSON response with the short code – I literally smiled at my screen.